

RustSim Geometry Module

The geometry module of the RustSim ecosystem handles coordinates, points in those coordinate systems, and regular one and two dimensional grids. Within a telescope model, the geometry module serves the following purposes:

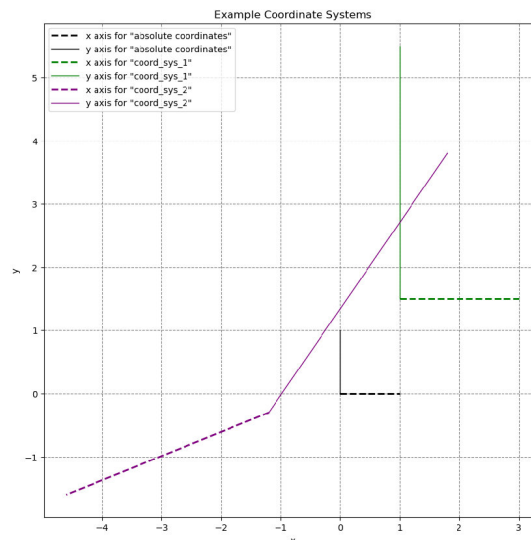
1. Allows the conversion between a location in the spacecraft field of view (FOV) and a location on a detector plane. This is needed to map a star from a location in the sky to a signal positioned on the detector and is a transformation effected by distortions and reflections from optical components.
2. Points in these coordinate systems are needed to represent the locations of data such as simulated or measured point spread functions (PSFs).
3. Spectral data such as filter curves are loaded into one dimensional regular grids, and spatial data such as vignetting profiles and spatially varying PSFs are loaded onto two dimensional grids. Loading one or two dimensional data allows us to interpolate the data to intermediate points. For example, for a given star we can interpolate between PSF shapes known at a finite set of locations to estimate a PSF at the exact location of the star.

Coordinate Systems

The field of view of the telescope and detector planes are examples of objects which require coordinate systems. A coordinate system is a collection of an x-axis, a y-axis, and a center. The usual cartesian coordinate system with center: $[0,0]$, x-axis: $[1,0]$, and y-axis: $[0,1]$ serves as the base coordinate system with respect to which all other coordinate systems are defined. This coordinate system is called absolute coordinates, and all points in any coordinate system are plotted in absolute coordinates. To create a new coordinate system we need to specify its center and x and y axis in terms of absolute coordinates. The center is given by an array $[x_o, y_o]$ which give the x and y position of the center of the new coordinate system in terms of absolute coordinates. The x and y axis are each given by an array of absolute coordinates $[x, y]$. If we specify an axis via $[x, y]$ this means that the unit vector of our new coordinate system will start at $[x_o, y_o]$ and end at $[x_o + x, y_o + y]$. A coordinate system does not have to be orthonormal, the only restriction is that the values used to specify it are Regular (not Nan or inf) and that the length of each axis is greater than 0. As an example we define the following two coordinate systems:

```
let coord_sys_1 = CoordinateSystem::new([2.,0.], [0.,4.], [1.,1.]);
let coord_sys_2 = CoordinateSystem::new([-3.4,-1.3], [3.,4.1], [-1.2,-0.3]);
let mut plot = Plot::new();
coord_sys_1.plot(&mut plot, 'green', 'coord_sys_1');
coord_sys_2.plot(&mut plot, 'purple', 'coord_sys_2');
plot.legend();
plot.set_title("Example Coordinate Systems");
plot.show('plot_path').expect('failed to plot coordinates')
```

Running this we get the following plot at 'plot_path':

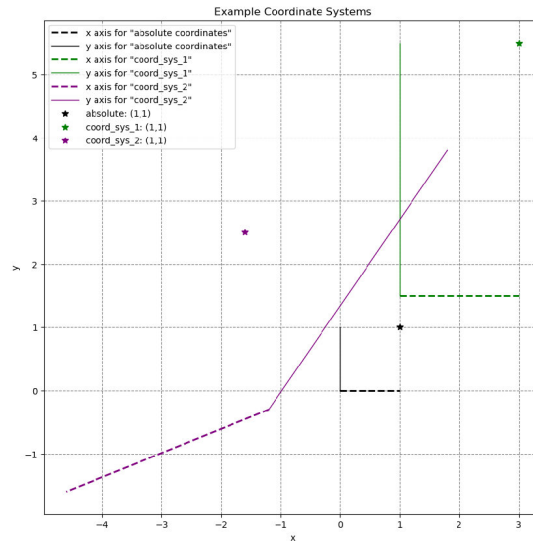


Points

To define a point we specify its x and y values with respect to any coordinate system. Here we make three different points each at position (1,1) in their respective coordinate systems:

```
let point_1 = Point::new(1.,1., &coord_sys_1);
```

```
let point_2 = Point::new(1.,1.,&coord_sys_2);
point_0.plot(&mut plot, 'black', 'absolute: (1,1)');
point_1.plot(&mut plot, 'green', 'coord_sys_1: (1,1)');
point_2.plot(&mut plot, 'purple', 'coord_sys_2: (1,1)');
```



Once defined in one coordinate system, points can be transformed into other coordinate systems with the "transform_to" method. 'to_absolute()' is shorthand for .transform_to(&ABSOLUTE_COORDINATES). Based on our definitions of 'coord_sys_1' and 'coord_sys_2' we expect that 'point_1' and 'point_2' will land at (3.,5.5) and (-1.6,2.5) in absolute coordinates respectively. 'point_0' should be unchanged. We can verify this by using the .hypot method to return the euclidean norm of the difference between the expected and actual coordinates:

```
let expected_0 = Point::new(1.,1.,&ABSOLUTE_COORDINATES);
let expected_1 =Point::new(3.,5.5,&ABSOLUTE_COORDINATES);
let expected_2 = Point::new(-1.6,2.5,&ABSOLUTE_COORDINATES);
//Two equivalent expressions:
assert!((point_1.transform_to(&ABSOLUTE_COORDINATES)-expected_1).hypot() < 0.00001);
assert!((point_1.to_absolute()-expected_1).hypot() < 0.00001);
assert!((point_0.to_absolute()- expected_0).hypot() < 0.00001);
assert!((point_2.to_absolute()-expected_2).hypot() < 0.00001);
```

Points can be transformed from one coordiante system to any other coordinate system with the .transform_to() method.

```
let point = Point::new(0.5,1.3, &coord_sys_1);
//hsekljfr
let transformed_point = point_1.transform_to(&coord_sys_2);
```

Regular Grids

Data used in the telescope simulator will need to be loaded into regular grids. For example, we require spectral responses to have been sampled at regular spectral intervals. Spatial data such as vinietting also must be given at regular intervals. Spectral data is loaded into agrid1d object, and spatial data is loaded into a grid2d object. Each spatial grid must be definedwith respect to some coordinate system and each spectral grid must be definedwith respect to a unit system

One Dimentional Grids

A one dimentional grid can be created with the following information:

1. The number of points in the grid.
2. The minimum value present in the grid.
3. The maximum value present in the grid,
4. The relative snap precision of the grid.
5. Units of the grid values.

The relative snap precision is the error to which the grid is tolerant to, given as a fraction of the grid step size. The snap precision used in calculations is the relative snap precision multiplied by the step size. For example, suppose that a grid has step size of 2nm with a grid point located at 1nm. If this grid had a relative snap precision of 0.001 then it's snap precision would be 0.002nm. A data point located at 1.0001nm would then be 'gridded' to the 1nm data point, since it is 'close enough' to 1nm. If the relative snap precision was 0.00001 however, such a point would not be 'gridded' to the 1nm data point. The snap precision is the tolerance of the

regular grid to numerical error. If you have a data set of values which should be a regular grid, but due to floating point error are not exactly regular you can raise the relative snap precision until the grid accepts them as regularly spaced within the expected tolerance. However, keep the precision of the grid as small as possible for best results when interpolating data. Grids can be defined with the `.new()` method, or by passing a vector of points:

```
let grid1d_1 = GRID1D::new(6, -1.5,3.5,0.001,Units::mm);
let points = vec![-1.5002,-0.4999,0.5008,1.5009,2.4997,3.4997];
let grid1d_2 = GRID1D::from_values(points,0.001,Units::mm);
assert_eq!(grid1d_1,grid1d_2);
```

The `grid1d` struct has is built with interpolation in mind. Given any point represented by a float and it's associated units,we can use the `.locate()` method to find where it is in the struct.

```
let point_2 = 3.50001;
let point_3 = -3.;
let point_4 = 5.;
match grid1d_1.locate(point_1,Units::mm) {
    Location::Higher => {
        println!('point is more than snap_precision higher than any grid point')
    }
    Location::Lower => {
        println!('point is more than snap_precision lower than any grid point')
    }
    Location::Snapped(i) => {
        println!('point is less than snap precision away from grid point i')
    }
    Location::Between(i, j, f) => {
        println!('point is between grid points {i} and {j}')
        println!('the distance between point and grid point {i} is {f} as a fraction of step
size')
    }
};
assert_eq!(grid1d_1.locate(point_1,Units::mm),Location::Between(0,1,0.5));
assert_eq!(grid1d_1.locate(point_2,Units::mm),Location::Snapped(5));
assert_eq!(grid1d_1.locate(point_3,Units::mm),Location::Lower);
assert_eq!(grid1d_1.locate(point_4,Units::mm),Location::Higher);
```

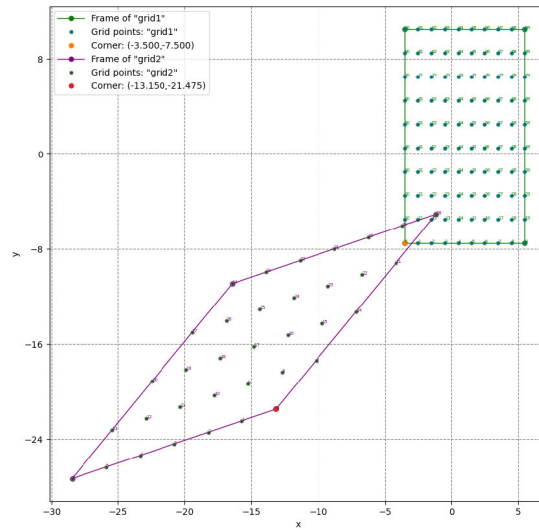
This location function is what becomes one dimensional interpolation when we move to data grids. Here the `Between` enum holds a scaled residue which is the fraction of the grid's step width that the point sits at away from the lower grid index it is between.

Two Dimensional Grids

Two dimensional grids are the structure behind spatially varying data.A two dimensional grid is specified by:

1. `[x_num, y_num]`: number of grid points in the x and y directions
2. `[x_step_size, y_step_size]`: distance between grid points in the x and y directions
3. `center`: Point struct which indicates the location of the grid. The point can be in any coordinates
4. `relative_snap_precision`: Just as for one dimensional grids.
5. `coordinate_system`: coordinate system of the grid in which the step sizes are calculated in

```
[10,10],
[0.5,0.5],
Point::new(0.,0.,&coord_sys_1),
0.0001,
coord_sys_1);
let grid2 = GRID2D::new(
    [7,5],
    [0.75,1.],
    Point::new(-16.,-16.5, &ABSOLUTE_COORDINATES),
    0.0001,
    coord_sys_2
);
let mut plot = Plot::new();
grid1.plot(&mut plot, 'green', 'grid1');
grid2.plot(&mut plot, 'purple', 'grid2')
plot.save('plot_path').expect('Could not plot grids');
```



Once a two dimensional grid is defined we can use the `.fit_grid()` method to fit the grid to an arbitrary point. On success the method returns a tuple `(x_index,y_index, x,y)`. The method takes a boolean value `fractional_residue`. If true, the function returns the residue of the point position relative to the grid point `(x_index,y_index)` as a fraction of step size. If false, `x,y` are float values in the coordinates of the grid. The fit grid function expects that the given point is within the grid (up to the grid snap_precision), If you wish to obtain the fit of a point projection